Problem Description

Alice is writing a multi-dimensional array template class `template<class T, int N> MultiArray`, where T represents the data type it stores, and N represents its number of dimensions. Your task is to help her progressively implement its features by completing the following subtasks in order.

Subtask 1: Initialization (20 points)

First, we need to implement the following basic functionalities for the multi-dimensional array:

- `MultiArray()`: Creates an empty N-dimensional array (where the size of each dimension is 0).
- `MultiArray(const vector<int> &dims, const vector<T> &data = vector<T>())`: Given dimensions `dims` and data `data`, creates an N-dimensional array. The size of each dimension corresponds to the elements of `dims` (it is guaranteed that each element of `dims` is a positive integer and its length is not greater than N). The elements of the array are filled with the elements from `data` in an order from lower to higher memory addresses (it is guaranteed that the length of `data` is the product of the elements in `dims`). For example, `MultiArray<int, 2> a({2, 2}, {1, 2, 3, 4})` creates a 2D array `a` where both dimensions have a size of 2, and the elements are assigned as `a[0][0] = 1, a[0][1] = 2, a[1][0] = 3, and a[1][1] = 4`. Additionally, this method must support the following special cases:
  - If `data` is empty (i.e., the length of `data` is 0, same below), all array elements should be initialized to `T()`.
  - If `dims` is empty, the size of the 0-th dimension is set to the length M of `data`, and the size of all other dimensions is set to 1.
  - If `dims` is not empty but its length is less than N, the initial dimensions are set according to the elements of `dims`, and the remaining unspecified dimensions are set to 1.
- `void set(const vector<int> &idx, const T &val)`: Sets the value at the position specified by `idx` to `val` (it is guaranteed that each element in `idx` is less than the corresponding dimension's size and that the length of `idx` is N).
- `T& get(const vector<int> &idx)`: Returns a reference to the value at the position specified by `idx` (the constraints on `idx` are the same as above). For example, for the 2D array `a` mentioned above, if `idx = {1, 2}`, it returns a reference to `a[1][2]`.
- `vector<int>& get_dims()`: Returns a `vector<int>&` of length N, representing the sizes of the current array's dimensions.

Subtask 2: Basic Operations (20 points)

Next, we want the multi-dimensional array to support the following basic operations:

- Support for `==`, `!=`, and `=` operators for the `MultiArray` type:

    - Two `MultiArray` objects are considered equal if and only if their dimension sizes and all corresponding element values are identical. They are considered unequal if there is any difference. Comparing uninitialized arrays is undefined behavior and does not need to be implemented.
    - The assignment operator `=` is only permitted between two multi-dimensional arrays of the same dimension sizes (or one is initialized only through `MultiArray()`). It is equivalent to assigning each element to the same position.
- Support for the stream insertion operator (`<<`) to print the entire array. The printing order should be from lower to higher memory addresses, as shown in the example below (No line break at the end, as shown in the public test case):

```
[0][0][0] = 1
[0][0][1] = 2
[0][0][2] = 3
[0][1][0] = 4
...
[3][4][2] = 114
```

Subtask 3: Iterator Implementation (20 points)

Implement a simple `Iterator` class for the multi-dimensional array that supports the following operations:

- Dereference operator `*`
- Prefix and postfix `++` operators, `==`, `=`, `!=` operators
- `begin()`: Returns an iterator pointing to the 0-th element of the corresponding multi-dimensional array.
- `end()`: Returns an iterator pointing to the element after the last element of the corresponding multi-dimensional array.

Subtask 4: Subscript Access (20 points)

Subtasks 4 and 5 are independent. You can skip Subtask 4 and complete Subtask 5 first.

Unlike the `T& get(const vector<int> &idx)` method, which accesses data elements using a `vector<int> idx`, in this stage, we want to access data elements directly using subscripts. For example, for `MultiArray<int, 2> A({2,2},{1,2,3,4})`, we need to support `A[0][1]` returning a reference to the element with the value 2. That is, the following code should be supported:

```
MultiArray<int, 2> A({2,2},{1,2,3,4});
A[1][1] = 10;
cout << A[1][1] << endl;
```

Subtask 5: Dimension Transformation (20 points)

Finally, we need to support two common dimension transformation methods for the multi-dimensional array:

- `reshape(const vector<int> &dims)`: Changes the dimensions of the array to `dims` (guaranteed that the length of `dims` is N). The elements in memory (from lower to higher addresses) remain unchanged. If the product of the elements in `dims` is not equal to the current total capacity of the array, this operation should be ignored.
- `resize(const vector<int> &dims)`: Changes the dimensions of the array to `dims` (guaranteed that the length of `dims` is N). For each position in the resized array, if its corresponding index existed in the original array, its value is the same as the value at that index in the original array; otherwise, its value is initialized to `T()`. For example, after performing `a.resize({1, 2, 3})` on `MultiArray<int, 2> a({1, 3, 4}, {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12})`, it becomes `MultiArray<int, 2> a({1, 2, 3}, {1, 2, 3, 5, 6, 7})`.
- Additionally, in this subtask, we will check the memory usage of your program. You must ensure there are no memory leaks to get full points.

Problem Requirements

Given `main.cpp`, `node.h`, and `array.h` (contents available via the download link), complete the following requirements:

- Do not modify `main.cpp` or `node.h`. `node.h` defines a custom fraction class and is only used as the data type for the multi-dimensional array in **Subtask 5**. You do not need to be concerned with its specific implementation.
- Implement all functionalities of the multi-dimensional array in `array.h`. You can implement them directly in this header file or other `*.cpp` files. We have predefined some methods of the `MultiArray` class in the provided files; you can add more if needed.

Grading Rules

The subtasks in this problem are **not** bundled for testing. Specifically:

- The first 4 subtasks each contain 1 public test case and 1 hidden test case, each accounting for 50% of the subtask's score.
- The 5th subtask contains 2 public test cases and 2 hidden test cases, each accounting for 25% of the subtask's score. 50% of these test cases will check for memory leaks.
- **All public test cases are identical to the provided files.** The scale of the multi-dimensional arrays in all test cases is small enough that you do not need to be particularly concerned with the execution speed of your program. In addition, we ensure that all operations are legal and there is no need to handle exceptions.

Input/Output Samples

This problem has no input files. The `main.cpp` files corresponding to the public test cases are located in the provided files as `main_1.cpp` to `main_6.cpp`, and their corresponding standard outputs are `1.out` to `6.out`. Public test cases 1-4 correspond to Subtasks 1-4, and 5-6 correspond to Subtask 5. For testing, you can rename the corresponding provided file to `main.cpp` to use a unified `Makefile` to generate the

executable file.

Hints

- **For Subtask 4**, assignment and stream operations require values of different types, so you might need to implement two versions of the `[]` operator.
- The standard output for some public test cases may be long. For Windows environments, you can use the `fc file1 file2` command in `cmd` to automatically compare two output files. For Mac and Linux, you can use the `diff file1 file2` command to do the same thing.

Submission Format

Please package `array.h`, `Makefile`, and any other `*.cpp` files you have implemented into a single zip archive, and upload it. Any other files in the submission will be ignored. The provided Makefile must be able to generate an executable named `main` (without an extension). **Note: Your files should be in the root directory of the zip archive, not in a subfolder. In other words, after unzipping your submission, it should directly yield a series of `*.cpp` files, `*.h` files, etc., not a folder containing them. During evaluation, the OJ will place the provided files into your directory.**

Disclaimer

While rigorous efforts have been made to ensure accuracy, technical translations may unavoidably contain errors or nuances that deviate from the original Chinese exam questions. **In the event of any discrepancy between the English translation and the original Chinese text, the Chinese version shall prevail as the authoritative reference.** Users are advised to verify critical terminology or ambiguous points against the Chinese source.